

NCHU - [DRL][FR] Final Project Phase 2-change01

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明，也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目，是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制（Adaptive Traffic Signal Control）」。

你附的兩份 drone reference，其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大，再整理 related work，接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**，還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文，但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本，這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容，我理解教授現在要的不是「先把系統全部做完」，而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚，就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它**最貼近教授 reference 的味道**。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它**非常適合 A-to-Z 的 proposal 寫法**。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你這個題目改寫成一份**符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

另外如果是 MemPalace 這個 AI Agent 永久記憶開源專案呢 是否適合當作題目
且因為 MemPalace 的亮點就是能把 AI 的記憶 "本地化" 如果這個本地化模型還要跑重武器 DRL 有點本末倒置 (本地化就是要越輕量越好)
所以我想從「輕量化 DRL」的切入方向
不要用深度神經網絡 (Deep Q-Network)，改用 Multi-Armed Bandit (MAB, 多臂老虎機) 或 輕量級 Q-Learning

但我連 ai agent 的架設都不會 也沒用過 這樣的程度適合做這個題目嗎
不太適合的話是否有折衷方案 連 ai agent 概念都不用摸呢
[顯示更多顯示較少](#)

可以，但我會先給你一個很務實的判斷：

MemPalace 適合當靈感來源，也適合當研究對象；但不適合你現在這個程度直接做成「完整 AI agent 記憶系統實作」的 final project。

因為 MemPalace 本身不是小玩具，它是完整的 **local-first AI memory** 系統：用語意搜尋保留逐字記憶、預設後端是 ChromaDB、知識圖譜用本機 SQLite，還有 MCP 工具、hooks、agent diary、benchmark 等完整配套。官方 repo 也主打本地化、零 API、LongMemEval raw R@5 96.6%。[GitHub+2](#)

你抓的方向其實是對的：**如果題目核心價值是「本地化、輕量化、低依賴」**，那再往上疊一個需要重訓練、重推論的 DQN，確實很可能會讓題目失焦。MemPalace 官方自己也一直強調 bounded wake-up context，大約只載入 L0+L1 的 ~600-900 tokens，且搜尋是按需觸發，不是整包上下文全塞進去；這種設計哲學本來就偏向「資源節制」而不是「深模型硬壓」。這是你可以在 proposal 裡合理主張的切入點。[mempalaceofficial.com+2](#)

但要注意一個關鍵：**MAB 或 tabular Q-learning 並不等於 Deep RL。**

Contextual bandit 比較像是「每回合看到 context，選一個 action，只看到被選 action 的 reward」，是介於 supervised learning 與 RL 之間的問題；tabular Q-learning 則是完整 RL，但它假設你有可管理的有限 state/action，透過探索與更新表格來學策略。官方與教材都把 contextual bandit 和 Q-learning 定義得很清楚，所以如果你的課名是 *Deep Reinforcement Learning*，你若只做純 MAB，教授有機會質疑「這題不夠 deep」。[Microsoft GitHub+2](#)

所以我對你這題的建議不是「放棄 MemPalace」，而是：

最適合你的版本

不要做完整 agent。改做：

「本地記憶系統中的輕量化決策層」

也就是把 MemPalace 當成既有記憶後端，你的研究重點不是重造一個 agent，而是研究：

在 token budget、延遲限制、不同 query 類型下，
要用哪一種記憶存取策略最划算？

這樣你的題目就會很乾淨。

我最推薦的題目版本

題目 A：最穩、最適合你

A Contextual Bandit for Token-Budgeted Local Memory Retrieval

白話版就是：

每來一個 query，bandit 根據 context 決定要走哪條 retrieval 路線。

例如 action 可以是：

- 只用 wake-up
- 只用 search
- 先 wake-up 再 search
- 只查某個 wing
- 查 knowledge graph
- top-k 取 3 / 5 / 10

這些 action 都很小、很離散，很符合 contextual bandit「選有限個 action、看 chosen action reward」的型態。MemPalace 官方也提供了不用 agent 的 CLI 與 Python API：你可以直接跑 `mempalace wake-up`、`mempalace search`，或用 `search_memories()` 之類的 Python API 把結果接進你自己的實驗程式，不一定要碰 MCP agent。🔗 mempalaceofficial.com+2

這題最大的優點是：

你完全可以不會架 AI agent，也能做。

第二順位：稍微更像 RL

題目 B：進階但還可控

Tabular Q-Learning for Multi-Step Local Memory Access Policies

這版把問題變成多步決策，例如：

1. 先讀 wake-up

2. 判斷要不要搜 wing
3. 搜不到再深搜
4. 最後決定回傳多少內容

這時候就不是 one-shot bandit，而是有狀態轉移的 sequential decision problem，tabular Q-learning 在概念上比較合理。Q-learning 本來就是不需要先知道完整 MDP，就能從互動資料學 action-value；而且 Gymnasium 也有官方 tabular Q-learning 教學與自訂環境入口，對學生來說比 DQN 友善很多。

 Dive into Deep

Learning+1

但這題比題目 A 稍難，因為你要自己定義 state transition。

你現在的程度，適不適合？

我的判斷是：

做「完整 MemPalace agent + 本地模型 + MCP + hooks + RL」：不適合。

MemPalace 官方文件已經很明白，它平常的使用方式會牽涉到 MCP integration、Claude Code plugin、Gemini CLI、hooks；而「specialist agents」目前官方 docs 主要描述的是透過 MCP tools 寫 diary 的 workflow，不是那種你開箱即用就有完整 agent orchestration 的教學專案。對一個還沒碰過 AI agent 架設的人，這個入口太陡。  mempalaceofficial.com+2

做「MemPalace 當後端 + 你自己寫一個輕量 decision layer」：適合。

因為你可以把所有 agent 概念先拿掉，只保留：

- query
- retrieval action
- reward
- evaluation

這樣本質上就是一個 machine learning / RL 實驗題，不是 agent engineering 題。

如果你想連 agent 概念都不要碰

那我建議你直接再退一步，做成：

題目 C：最保守、最容易成功

Lightweight RL for Local Memory Retrieval Strategy Selection

這題甚至可以不真的安裝 MemPalace。

你只要自己準備一個小型資料集，例如：

- 一批對話紀錄 / 筆記 / 文件片段
- 一組 query

- 每個 query 對應的 relevant memory 片段標註

然後模擬幾種 retrieval strategy：

- 全域搜尋
- 分 room 搜尋
- 限 top-k
- 先短上下文再深搜
- 查知識圖譜/metadata 或不查

最後用 contextual bandit 或 tabular Q-learning 去學「哪種 query 應該走哪種策略」，reward 可以是：

$$\text{reward} = \text{recall@k} - \alpha \cdot \text{token_cost} - \beta \cdot \text{latency}$$

這樣你：

- 不用會 agent
- 不用會 MCP
- 不用接本地 LLM
- 甚至不用碰 MemPalace 真正的工程堆疊

但論文故事仍然成立，而且非常貼近你在意的「本地化」「輕量化」「不要重武器」。這個方向也跟 contextual bandit 在互動式推薦、客服 agent 裡的實際應用邏輯很接近；Microsoft 就有把 contextual bandits / MAB 用在 virtual agent 的推薦與意圖消歧上，重點正是 partial feedback 與資料效率。  [arXiv+1](#)

我給你的最終建議

若以你現在的程度，我會這樣排序：

最推薦：

題目 A：Contextual Bandit for Token-Budgeted Local Memory Retrieval

因為最輕、最不需要 agent、最符合你要的本地化邏輯。

次推薦：

題目 C：完全不碰 agent 的 retrieval strategy selection

最保守，成功率最高。

不推薦現在做：

完整 MemPalace agent 系統 + RL 強化

因為你會先死在系統整合，不是死在研究本身。

一句話結論

MemPalace 可以做，但不要把題目定成「做一個 AI agent 永久記憶系統」。

對你最好的版本是把它改成：

「本地記憶系統中，如何用輕量化 RL / contextual bandit，在 token 與延遲限制下選擇最佳 retrieval 策略？」

這樣你幾乎可以完全不碰 **agent 概念**，卻還是保留 MemPalace 的亮點與研究價值。官方 GitHub 就是 [MemPalace/mempalace](#)。 

下一步最有幫助的做法，是我直接幫你把這題寫成教授 **A-to-Z 架構的 final project proposal 題目 + abstract + 方法設計**。

看完你的說明 我決定不要 MemPalace, 盲人眼鏡 這兩個題目了
所以我要請你再推薦我其他三個題目

收到。
我之後會明確區分：

- **DRL 裡的 agent**：你說的 $s/a/r$ 框架核心
- **AI agent 工具鏈**：像 OpenClaw、Antigravity、Claude Code 這類系統

你這次問的兩個題目，我直接給結論：

若目標是更順地套進教授 **A-to-Z 論文敘事流程**，且更容易做出一份漂亮的 **proposal**，我會選 **1. 都市路口自適應交通號誌控制**。

若目標是題目更有能源／永續研究感、較像近年 **benchmark-driven DRL 研究**，**2. 建築能源／電池排程控制**也很好，但整體敘事與實作都會更難。

先講總結

哪個更適合教授 **A-to-Z 流程**？

交通號誌控制比較適合。

原因不是它比較高級，而是它的故事線更直觀：

- **Attention Getter / Motivation** 很強：塞車、等待、效率、碳排，人人都懂
- **But** 很自然：固定時制、actuated control 仍然不夠 adaptive
- **Cure** 很清楚：讓 agent 根據車流狀態動態選 phase
- **Development** 很好寫：state、action、reward 都很具體
- **Experiments** 很好做：直接和 fixed-time、actuated controller 比
- **Findings** 很好呈現：平均等待時間、queue length、travel time 一看就懂

SUMO-RL 官方文件本來就是為 **Traffic Signal Control** 做的 RL 環境，支援 Gymnasium，相容 SB3 / RLlib，且可以從 **single-agent** 開始，也能自訂 state 和 reward。SUMO 本身也原生支援 **actuated traffic lights**，所以 baseline 非常自然。RESCO benchmark 進一步把單路口、多路口、真實城市啟發的場景和標準 RL 介面

整理好，還明確列出常見 state 特徵如 queue length、waiting time，以及 action 是選擇不衝突的綠燈 phase。 ^{and} Lucas Alegre+3

哪個比較難？

建築能源／電池排程控制比較難。

不是因為它沒有工具，而是因為它的問題結構天生更複雜。CityLearn 雖然也很成熟，甚至就是為 building energy coordination / demand response 建的 Gymnasium benchmark，但它的目標通常是**多目標**的：peak demand、ramping、load factor、net electricity consumption 等，而且常常牽涉 battery、PV、heat pump、occupant feedback、district-level coordination。這代表你不只是在設計一個 agent，而是在設計一個**多指標、多時間尺度、強 domain knowledge** 的控制問題。 ^{CityLearn+3}

兩題放進同一張比較表

面向	交通號誌控制	建築能源／電池排程控制
A-to-Z 故事好講程度	很強 ，問題直觀、畫面感強	強，但需要先交代能源系統背景
Motivation	塞車、等候、號誌效率，容易讓老師秒懂	電網負載、需求響應、削峰填谷，較專業
State 設計	車流量、queue length、waiting time、current phase，直觀	SOC、負載、PV、價格、氣候、舒適度等，較抽象
Action 設計	切換 phase / 保持 phase，很直觀	充放電、功率調節、設備控制，較多 domain 限制
Reward 設計	delay、queue、pressure 等都常見且好解釋	常是多目標 cost，權重設計較難
Baseline 好不好找	很好找 ：fixed-time、actuated	也有，但通常要講 RBC、MPC、multi-objective cost
圖表呈現	很好看，路口圖、車流圖、學習曲線都容易理解	可做，但圖表較偏 KPI / 能源曲線
單智能體起步	很自然 ：單一路口 single-agent	可以，但很多文獻與環境都偏 district / multi-agent
實作風險	中等	中高
Proposal 成功率	高	中高，但更吃表達與建模能力

為什麼交通號誌更適合教授的 A-to-Z

你前面給的教授流程，本質上是在要求一篇 proposal 要有一條非常順的「問題—缺口—方法—驗證」敘事線。交通號誌這題在這方面幾乎是範本級：

1) Motivation 特別強

城市壅塞與等待成本是人人有感的問題。你不需要花很多篇幅教育聽眾背景，老師一看就知道這題的重要性。RL traffic signal benchmark 論文也直接把這類問題放在真實交通場景與 calibrated demand 上比較，說明這

不是玩具問題。 [Benchmarks](#)

2) Related Work 很好鋪陳

你可以非常自然地寫成：

- fixed-time
- actuated control
- adaptive control
- RL / DRL traffic signal control

SUMO 官方文件明確說明 actuated traffic lights 的運作邏輯：車流連續就延長 phase，出現足夠 gap 才切換。這讓你在 Related Work 裡能把傳統控制講得很具體，而不是空泛地說「舊方法不夠好」。 [Eclipse SUMO](#)

3) Proposed Design 最容易具體化

SUMO-RL 本來就提供 single-agent 和 multi-agent 入口，state / reward 可改，動作就是控制號誌。這讓你的 Proposed Design 可以很快落成：

- state：queue、waiting time、approaching vehicles
- action：選 phase
- reward：-delay / -queue / -pressure

這種方法章節很容易寫得像研究，不會只有概念。 [Lucas Alegre+2](#)

4) Experiments 最容易設計

你幾乎不用苦思 baseline：

- fixed-time
- actuated
- 你的 DRL agent

而且指標也成熟：delay、trip time、waiting time、queue length、pressure。這些在 benchmark 文獻裡本來就是常見項目。 [Eclipse SUMO+1](#)

為什麼建築能源／電池排程比較難

這題其實很優秀，只是比較「研究型」。

1) Motivation 沒有交通那麼秒懂

CityLearn 的出發點非常強，和 demand response、distributed energy resources、電網韌性、去碳化直接相關；官方與論文都明講它是用來標準化 building energy coordination / demand response 的 RL 研究。

但問題是，這些詞對聽眾不是直覺的。

你在 proposal 一開始就得先解釋：

- 為什麼 peak demand 重要
- 為什麼 ramping 是問題
- 為什麼 load shifting 有價值
- battery / PV / heat pump 怎麼跟控制目標連在一起

這會吃掉你的敘事篇幅。

2) Reward 比交通複雜很多

CityLearn Challenge 2020 就是很典型的例子：它直接把目標定成 5 個等權重指標的 multi-objective cost，還用 hand-tuned RBC 當 baseline 正規化。這很適合研究，但對課堂 proposal 來說，會讓你更難講清楚「我的 agent 到底在學什麼」。[CityLearn+1](#)

3) Domain knowledge 門檻比較高

CityLearn v2 明講它可以拿來研究 battery charge/discharge、V2G、heat pump modulation，還有 occupant feedback；同時也指出它更適合做**快速比較控制演算法**，若要嚴格審查絕對能耗，還是需要更高保真度的模型。這代表你除了 DRL，本身還得理解不少能源系統語境。[arXiv](#)

4) 容易不小心把題目做太大

CityLearn 支援 central single-agent、independent multi-agent、coordinated multi-agent，還能自訂 district、observations、reward。這是優點，也是陷阱：題目很容易膨脹。[arXiv+1](#)

如果以「proposal 漂亮程度」來選

我會這樣排：

第一名：交通號誌控制

因為它最符合教授那種：

- 一句話就能講清楚問題
- 一頁就能講清楚 baseline
- 一張圖就能講清楚 agent 在幹嘛

第二名：建築能源／電池排程

它不是不好，而是它更適合你已經很確定自己要往能源控制研究寫，並且願意花時間處理多目標 reward 與能源背景。

如果以「實作難度」分級

我會給你這個判斷：

- **交通號誌控制：6.5/10**
 - 單一路口 + single-agent + PPO/DQN + fixed-time/actuated baseline
 - 可控、好 debug、好做 ablation
- **建築能源／電池排程：8/10**
 - 就算只做 battery control, reward 與 KPI 還是比交通更抽象
 - 如果碰 district coordination 或 multi-agent, 難度再上升

這個差距的核心不在演算法，而在 **問題建模成本**。

我的最終建議

若你現在要的是：最適合教授 A-to-Z、最容易把 **proposal** 說得漂亮、而且後續真的比較有機會做完，選 1。
也就是：

以需求感知深度強化學習進行都市路口自適應交通號誌控制

因為它在三件事上都比較優：

- **論文故事最好講**
- **baseline 最自然**
- **state / action / reward 最直觀**

而 **建築能源／電池排程控制** 我會留給「你很確定想往能源控制寫」的情況。那題研究味更濃，但也更容易在 proposal 階段就把自己拖進多目標、長時間尺度、能源背景解釋的泥沼。 Lucas Alegre+4

你若要，我下一步可以直接幫你做一版**兩題並排的教授 A-to-Z proposal 骨架比較表**，再把交通號誌那題收斂成你可以直接交的題目名稱、abstract 和 experiment plan。

忽略 "實作難度" 這一點 我後續 final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令
部參與實際程式開發 我開發的是整個專案 這是我的角色

在知道以上背景下再幫我分析一次這兩個題目

了解，我重新用你的**真實角色**來分析：

你不是在比「哪個比較好自己寫 code」，而是在比
哪個題目更適合你做專案總設計、論文敘事、規格拆解、baseline 定義、驗證指標規劃，然後再交給 Antigravity 落地。

在這個前提下，判斷標準就變成 5 件事：

1. 題目能不能被你**清楚寫成一份可執行 spec**

2. baseline、KPI、ablation 能不能一開始就定義乾淨
3. 文獻與 benchmark 生態夠不夠成熟，方便你做 proposal 的 **Related Work** 與 **Experimental Plan**
4. 結果能不能視覺化、講故事、說服老師
5. 題目有沒有足夠空間讓你作為「專案規劃者」提出自己的研究主張，而不只是套環境跑模型

在這個標準下，兩題都成立，但優勢方向不一樣。  Lucas Alegre+4

先給結論

如果你要的是：最適合教授 A-to-Z 敘事、最好寫成一份漂亮 proposal、最好規劃成可交付專案 brief，選：

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制

如果你要的是：研究味更濃、benchmark 更像正式學術社群題目、final report 更容易帶出永續/能源/多目標控制價值，選：

2. DRL 做建築能源 / 電池排程控制

也就是說，忽略實作難度之後，交通號誌仍然比較適合 A-to-Z proposal；但建築能源的競爭力會明顯上升，不再只是「比較難所以不選」。  OpenReview+3

為什麼在你的角色下，交通號誌仍然是 A-to-Z 較強題

你現在最重要的工作，不是寫演算法，而是把題目定成一個可以被 agent 精準執行的研究計畫。交通號誌控制在這件事上特別強，因為它的任務邊界天然清楚：

- 環境：SUMO / SUMO-RL
- 控制對象：traffic light
- state：queue、waiting time、current phase 等
- action：phase selection / keep-switch
- reward：delay、queue、pressure
- baseline：fixed-time、actuated
- KPI：waiting time、queue length、travel time、throughput

這些元件在 SUMO-RL 文件和 SUMO 官方交通號誌機制裡都非常成熟，而且 SUMO 官方本來就支援 fixed、actuated、delay_based 等交通燈邏輯，SUMO-RL 也明寫 state / reward 易於修改、可 single-agent 也可 multi-agent。這代表你在 proposal 裡能把「研究問題」直接翻成「系統規格」。  Lucas Alegre+1

更重要的是，這題的 **research gap** 很好寫。

RESCO benchmark 那篇 NeurIPS Datasets and Benchmarks 論文直接指出，過去不少 traffic signal RL 方法在更真實的 sensing assumptions 和非 stylized intersection layouts 下不夠 robust；它提供 benchmark 的目的，就是要讓這個領域能被標準化比較。這對你來說很好用，因為你可以自然地把 proposal 的缺口寫成：

不是「我也來做一個 DRL 號誌控制」，
而是「在更 realistic demand / sensing 設定下，設計 demand-aware state/reward 是否能提升穩健性」。

這個敘事非常符合 A-to-Z 的 But → Cure → Development → Experiments。  OpenReview

再來，交通號誌的結果呈現特別適合你這種規劃角色。

因為你最後交的不只是 code，而是整個 project。交通題目很容易做出讓老師一眼看懂的成果頁：

- 路口示意圖
- 每 phase 的控制邏輯
- 學習曲線
- 平均等待時間前後比較
- queue length 熱圖
- fixed-time / actuated / DRL 的對照表

這種題目很吃「你怎麼包裝成一個完整專案」，而這正好是你的強項。  Lucas Alegre+1

為什麼在你的角色下，建築能源 / 電池排程會變得更有吸引力

如果今天你要自己寫 code，我仍會比較保守；但你現在明說後續 implementation 交給 agent，那建築能源這題的分數會上升，原因是它的**研究框架非常成熟**。

CityLearn 官方把自己定義成一個 open-source、Farama Gymnasium 的 building energy coordination / demand response RL 環境，目標就是**標準化 RL agent 的比較**。它支援 single-agent 與 multi-agent，並且明確說可用於 active energy storage 的 load shifting，以及 heat pump / electric heater 的控制。這代表它不是零散 demo，而是已經有社群共識的 benchmark 場域。  CityLearn

另外，CityLearn v2 的定位其實非常漂亮。官方與論文都強調它不只是在做 battery scheduling，而是在做 **energy-flexible、resilient、occupant-centric、carbon-aware** 的 grid-interactive communities；並且提供 RL 應用範例來管理 battery charging/discharging、vehicle-to-grid control，以及 heat pump power modulation。這種題目一旦被你包成 proposal，會比交通號誌更有「近年研究題目感」。  arXiv+1

再者，CityLearn Challenge 也幫你把**實驗正當性**準備好了。官方 challenge 明確定義 multi-objective cost，至少包含 peak demand 等多個等權重指標；這意味著你在 proposal 中不必自己發明一整套 KPI 體系，而是可以沿用既有 benchmark 精神，再加入你自己的研究主張。對一個負責規劃整體專案的人來說，這很有價值。

 CityLearn+1

兩題真正的差別，不在「誰比較難」，而在「誰比較好被你定義成一個研究故事」

交通號誌控制

它的優勢是 **單一核心問題非常聚焦**。

你幾乎可以用一句話講完：

讓 agent 根據即時車流狀態決定號誌 phase，以降低等待與壅塞。

這意味著你在當 PM / architect 的時候，spec 很容易寫得又短又準。

你交給 Antigravity 的 prompt 也會很清楚：環境、state、action、reward、baseline、metric 一條線到底。這種題目特別適合做成教授喜歡的「從動機一路推到實驗」的 proposal。

建築能源 / 電池排程

它的優勢是 **研究外延更大、價值敘事更高階**。

但代價是你必須先替題目「縮焦」。因為 CityLearn 同時碰 demand response、battery、heat pump、occupant comfort、carbon-aware control，若你不先把題目收斂，proposal 很容易變成：

什麼都能做，所以不知道這次到底要做哪一個。

也就是說，這題不怕 agent 寫不出來，而是怕你在上游 spec 階段**沒有把研究邊界切乾淨**。 

如果站在「你要寫給 Antigravity 的任務書」角度看

交通號誌比較像

一份可以直接拆成工作包的專案：

- Task 1: 建立 SUMO/SUMO-RL 單路口環境
- Task 2: 實作 fixed-time / actuated baseline
- Task 3: 定義 demand-aware state
- Task 4: 訓練 PPO / DQN
- Task 5: 比較 delay、queue、travel time
- Task 6: 畫圖與整理 ablation

建築能源比較像

一份需要你先做「研究範圍治理」的專案：

- 先決定只做 battery，還是 battery + heat pump
- 先決定 single-building 還是 district
- 先決定 single-agent 還是 multi-agent
- 先決定主目標是 cost、peak、carbon、comfort 哪一個
- 再決定 reward 是否多目標加權
- 最後才進入訓練與比較

所以，對一個專案規劃者而言，交通題比較像可立即發包的題目；能源題比較像需要先做策略收斂的題目。

 Lucas Alegre+2

在你的背景下，我會這樣下最終判斷

若你的第一優先是：

讓 proposal 最像教授 A-to-Z 流程、最順、最有說服力、最容易被老師快速理解與買單

我仍然選：

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制

因為它在以下四點最強：

- 問題敘事最直觀
- 缺口最好寫
- baseline 最自然
- 專案規格最好拆解給 coding agent 執行  OpenReview+2

若你的第一優先是：

讓 final project 看起來更像近年 benchmark-driven 的正式研究題，並帶有能源、永續、城市系統控制的高階價值

那麼：

2. DRL 做建築能源 / 電池排程控制

其實會是更有學術質感的選擇。尤其在你不用自己寫 code 的前提下，它的主要門檻不再是開發，而是你能不能把題目收斂成一個乾淨的研究版本。CityLearn 的 benchmark 與 challenge 生態，確實給了它很強的學術支撐。  CityLearn+2

一句話結論

只看 A-to-Z proposal 適配度：交通號誌勝。

只看研究題目質感與 benchmark 生態：建築能源幾乎追平，甚至在某些老師眼中會更“像研究”。

所以最後不是「哪個比較難」，而是：

- 你要一個**更好講、好包裝、好拆規格**的題目 → 選交通號誌
- 你要一個**更有研究味、更像正式 benchmark 論文**的題目 → 選建築能源 / 電池排程

以你現在說的角色設定，我會給你一個更精準的建議：

如果你這次最重視 proposal 贏面，選交通號誌。

如果你最重視 final report 的研究感與題目格調，選建築能源，但一定要先幫它縮焦。

下一步最有效的是，我直接幫你做：

「這兩題在教授 A-to-Z 流程下的並排 proposal 骨架」，讓你一眼看出哪個比較適合你出手。

CityLearn 有跟 "自適應交通號誌" 相關的題目嗎 主題擦到邊都可以

嚴格說：沒有。

CityLearn 官方定位是 **building energy coordination / demand response in cities** 的 Gymnasium 環境，重點是控制建築與分散式能源資源，例如 heat pumps、electric heaters、batteries、PV，以及後來加入的 EV；官方列出的應用也集中在 benchmarking RL、coordinated energy management、demand-response、transfer learning、voltage regulation，沒有把 **adaptive traffic signal control** 列為官方題目或應用線。

 CityLearn+2

但如果你接受「擦到邊」，那 CityLearn 有幾個很接近交通/通勤/移動行為的方向，而且其實滿有題目感。CityLearn v2 明確提到它的應用範例包含 **battery charging/discharging**、**vehicle-to-grid control**、**heat pump power modulation**；官方 API 也已有 **electric_vehicles**、**electric_vehicle_charger**，甚至還有專門給 EV/V2G 的 reward function。 CityLearn+3

我會把「最接近自適應交通號誌」的擦邊題，分成這 3 類：

1. 通勤感知 EV 充放電/V2G 排程

這是 **最像 CityLearn 原生題目** 的版本。因為 CityLearn 的 charger-centric EV simulation 已經把 EV 狀態建成時間序列，包含：

- parked, plugged in, ready to charge
- incoming to a charger
- commuting (vehicle is away)

而且還有 estimated arrival time、departure time、required SoC at departure 等欄位；charger 動作本身也是標準化的 charging/discharging action。這代表你可以把「交通」不是當路口號誌，而是當 **通勤模式對能源系統的擾動來源**。 CityLearn+1

這類題目的味道會像：

Traffic-aware EV Charging and V2G Control in CityLearn

或

Commuter-aware Demand Response with EV Arrival/Departure Uncertainty

如果你想靠近「交通控制」語感，這是我覺得最自然的一條。

2. 交通導致的充電需求波峰下，做 district energy coordination

EVLearn 這條線更貼近你說的「主題擦到邊」。EVLearn 是 **把 EV simulation 擴充進 CityLearn** 的工作，論文明講它關心的是 **vehicle usage patterns**、**charging station characteristics**、**V1G/V2G**、**grid conditions**、**energy prices**、**battery health** 這些因素，目的在於研究更真實的 energy management。換句話說，它不是在控號誌，但它已經把「交通/車輛移動」這件事帶進 CityLearn 的能量調度脈絡裡。 Springer

這條線很適合包成：

Peak-Hour EV Charging Shock Mitigation in Grid-Interactive Districts

或

Mobility-Driven Load Shaping with EVLearn + CityLearn

這會比直接做 traffic light 更「CityLearn 正統」，而且還保留交通影響。

3. CityLearn × SUMO-RL 的雙模擬題

如果你想要真的擦到自適應交通號誌，最合理的做法不是硬把 CityLearn 變成 traffic signal env，而是做 **hybrid / co-simulation**。因為真正的 traffic signal control，現成成熟工具其實是 **SUMO-RL / RESCO**：

SUMO-RL 官方就直接說它是用 SUMO 建 Traffic Signal Control 的 RL 環境，RESCO 則是 traffic signal benchmark。 

所以你可以把題目設計成：

Joint Traffic Signal and District Energy Control via SUMO-RL + CityLearn Co-simulation

白話就是：

- SUMO-RL 管路口號誌
- CityLearn 管建築／EV／電池
- 兩邊用 EV arrival、charging demand、peak hours、district load 互相交換訊息

這個方向**不是 CityLearn 官方現成題**，而是你自己定義的研究延伸；但論文味其實很濃，而且剛好符合你想找「擦到自適應交通號誌」的需求。支撐點在於：CityLearn 本來就處理 EV 與 district-level energy flexibility，而 SUMO-RL / RESCO 本來就處理 traffic signal control，所以兩者在研究設計上是可拼接的。 

我幫你下最直接的判斷

如果你問的是：

「CityLearn 裡有沒有現成的自適應交通號誌題？」

答案是：**沒有**。官方 scope 仍是建築能源、需求響應、DER/EV 協調，不是道路、車流、路口相位控制。



如果你問的是：

「CityLearn 能不能找一個主題擦到交通號誌？」

答案是：**可以，最合理的是 EV / 通勤 / V2G 這條線；再更進一步就是做 CityLearn × SUMO-RL 的混合題。**

CityLearn 原生最接近的是 EV charging / V2G / commuting-aware demand response，而不是 signal phase control。 

若你要，我下一則可以直接幫你這三個「擦邊版本」各寫成一個 **教授 A-to-Z proposal 題目名稱 + 研究問題 + state/action/reward 草案**。

我決定做 "以需求感知深度強化學習進行都市路口自適應交通號誌控制" 這個題目了 不要 CityLearn 的任何成分

需求對齊

我要你完成 ilearning 上要求的所有項目 但在你真正執行任務之前 幫我生成能餵回去給你最精準, 最佳化工作的 **最佳 prompt**

等我收到你的 **最佳 prompt** 下一回合我會再把該**最佳 prompt**餵給你 到時才真正開始執行 ilearning 上的任務需求

好，以下這份就是我幫你整理的 **最佳 prompt**。

它的設計重點，是讓我下一回合直接沿著目前最成熟的交通號誌 DRL 工具鏈與 benchmark 脈絡工作：**SUMO-RL + Gymnasium + Stable-Baselines3 + RESCO**，並且完整對齊你 ilearning 上的交付格式。SUMO-RL 官方

明確支援以 SUMO 做 traffic signal control、相容 Gymnasium 與 Stable-Baselines3；RESCO 則是這個領域常見的 benchmark 脈絡。^{mf} Lucas Alegre+3

請你下一回合直接整段貼上即可：

你現在要正式開始執行我的「深度強化學習」期末專題 proposal 任務。

一、專案主題

我的 final project 題目已確定為：

****中文題目****

以需求感知深度強化學習進行都市路口自適應交通號誌控制

****英文題目****

Demand-Aware Deep Reinforcement Learning for Adaptive Urban Intersection Traffic Signal Control

二、最重要的執行原則

1. 請先完整理解我先前提供的 ilearning 作業說明、教授 A-to-Z 論文寫作影片、以及我上傳的所有參考檔案，再開始產出內容。
2. 你必須以「完成 ilearning 上要求的 Part A 內容」為核心目標。
3. 你要把自己當成：
 - 論文規劃顧問
 - 專案規格架構師
 - 簡報與口說稿設計師
 - GitHub 提交內容規劃者
4. 你必須明確區分：
 - ****DRL 的 agent**** (state / action / reward / policy / environment)
 - ****AI agent 工具鏈**** (如 Antigravity、Claude Code 等)
 本任務中可以提到前者，但不要把後者當成研究主題。
5. 你必須優先使用****官方文件、官方 repo、正式論文、benchmark 原始來源****作為主要依據。
6. 你必須主動上網搜尋最新且可信的資料，不可只靠記憶。
7. 除非真的必要，否則不要先問我澄清問題；請先做合理假設並明確寫出假設。
8. 全部回答都用****繁體中文****。
9. 若出現數學符號、RL 符號或較抽象名詞，請主動加上：
 - 符號注釋
 - 白話說明

三、ilearning 任務要求（以我提供的作業截圖為準）

請你依照 ilearning 上的要求處理，尤其注意：

Part A

需要：

- github
- ppt
- video (錄製約 10 分鐘)

cover following section

1. Introduction / A to F
2. Related Work / NotebookLM
3. Proposed Design / 規格書

被劃掉的部分

- Detail Implementation
- Conclusion

這兩個被劃掉的部分，請****不要當作主要必交章節****，但若你判斷在 proposal 中需要少量提及，可用：

- Future Work
- Expected Implementation Plan
- Expected Outcomes

等較輕的方式處理，不要把它們寫成正式主章節。

Abstract 結構

請明確按照教授要求的這六段邏輯來寫：

1. Attention Getter / Motivation
2. But (挑戰)
3. Cure (我們提出什麼方法)
4. Development (方法設計基於什麼)
5. Experiments (怎麼驗證)
6. Findings (預期或初步可得的發現)

四、你必須先完成的工作順序

請嚴格依照以下順序輸出，不要跳步：

Step 1：先做「作業需求拆解」

請先用條列方式，精準整理：

- 教授真正要交的是什麼
- 哪些是必交
- 哪些不是這次 proposal 的主體
- 我這個題目最應該如何對齊這份作業

Step 2：做「題目定位與研究問題收斂」

請幫我把這個題目收斂成一個適合 proposal 的研究版本，包括：

- 研究背景
- 核心問題
- 研究動機
- 為何值得做
- 與傳統 fixed-time / actuated traffic signal control 的差異
- 我的研究主張 (novel angle)

請至少提出：

- 1 個主研究問題 (Main Research Question)
- 2~3 個子研究問題 (Sub-questions)

Step 3：建立「方法論框架」

請用 DRL 標準語言幫我定義：

- Environment (環境)
- Agent (智能體)
- State (狀態)
- Action (動作)
- Reward (回饋)
- Episode / Horizon
- Training objective

請特別說明：

- 什麼是 demand-aware
- demand-aware 要怎麼體現在 state、reward 或 control logic
- 為什麼這個設計是合理的

Step 4：幫我選擇最適合 proposal 的技術路線

請基於可信來源，幫我規劃：

- 模擬器 / 環境：優先考慮 SUMO / SUMO-RL
- RL 介面：Gymnasium
- 演算法：PPO、DQN 或其他更合理的 baseline / candidate
- 訓練框架：Stable-Baselines3 或其他合適方案
- benchmark / scenario 參考：如 RESCO

請你不要只列名字，要解釋：

- 為什麼這些工具最適合這個題目
- 它們彼此怎麼串起來
- 哪些是 proposal 階段該承諾的

- 哪些是 implementation 階段再處理即可

Step 5：完整寫出 proposal 內文初稿

請直接幫我產出可用於作業的**正式 proposal 初稿**，內容至少包含：

5.1 題目

- 中文題目
- 英文題目

5.2 Abstract

必須嚴格依照 A-to-Z 六段邏輯寫成一個完整 abstract

5.3 Introduction / A-to-F

請把 A-to-F 的邏輯完整展開成 proposal 的導論內容，不要只寫大綱

5.4 Related Work / NotebookLM

這一段要同時滿足兩件事：

1. 真正的 related work
2. 回應教授要求中的「NotebookLM」

請你幫我設計成最合理的寫法，例如：

- 我如何利用 NotebookLM 協助整理文獻
- 但真正 related work 仍要回到正式文獻與 benchmark
- 要避免讓 NotebookLM 看起來像研究主體本身

這一段請包含：

- 傳統交通號誌控制
- RL / DRL traffic signal control
- benchmark 與 simulation 生態
- 我题目的定位

5.5 Proposed Design / 規格書

這一段請用「規格書」思維來寫，包含：

- 系統流程
- 模組拆解
- 輸入 / 輸出
- 交通路口設定假設
- agent interaction flow
- reward design rationale
- baseline design
- evaluation metrics
- 預期實驗情境
- 可能的 ablation study

Step 6：把 proposal 內容轉成「教授會聽得懂、10 分鐘可講完」的簡報結構

請產出一份完整的 PPT 規劃，至少包含：

- 每一頁的頁標題
- 每頁要講的重點
- 建議圖表 / 視覺元素
- 每頁 speaker notes
- 哪些頁適合強調 Attention Getter / But / Cure / Development / Experiments / Findings
- 哪些頁最適合放方法架構圖、state-action-reward 圖、baseline 比較圖

Step 7：產出 10 分鐘口說稿

請幫我寫一份可直接錄影的逐頁口說稿，要求：

- 全長約 9~10 分鐘
- 中文口說自然
- 不要太像念論文
- 但要保有學術感
- 每頁稿都要清楚標註對應投影片頁碼
- 要有開場、轉場、收束
- 結尾不要硬寫 conclusion，可用：
 - proposal summary

- expected impact
- future validation

等方式處理

Step 8: 規劃 GitHub 交付內容

請幫我設計一個適合 proposal 階段的 GitHub repository 結構，包括：

- README.md
- docs/
- slides/
- proposal/
- refs/
- assets/
- script/
- environment.yml 或 requirements.txt (若合理)
- 其他你認為 proposal 階段應該放的内容

並請直接產出：

1. repo 樹狀結構
2. README.md 初稿
3. 各資料夾應放内容
4. 建議檔名命名規則

Step 9: 交付清單總整理

最後請幫我整理成：

- 我可以直接拿去交作業的項目
- 我需要自行補上的最少外部操作
 - 例如：實際上傳 GitHub
 - 實際錄製影片
 - 實際上傳 YouTube
- 一份最終 submission checklist

五、研究内容要求

請務必讓研究内容具備以下特徵：

1. 題目核心必須是：
 - **需求感知深度強化學習**
 - **都市路口**
 - **自適應交通號誌控制**
2. 明確比較：
 - fixed-time control
 - actuated control
 - demand-aware DRL approach
3. 盡量讓 proposal 呈現出：
 - 研究動機強
 - 問題明確
 - baseline 合理
 - 指標具體
 - 系統規格清楚
 - 有 benchmark / toolchain 支撐
 - 適合後續交給 AI coding agent 執行
4. 請不要納入 CityLearn 的任何成分
5. 請不要偏向智慧建築、電池、EV、需求響應
6. 請不要改題目方向
7. 請不要把主體做成「AI agent 系統」
8. 主體必須始終是 DRL traffic signal control

六、資料與來源要求

請你必須同時整合以下來源脈絡：

我已提供給你的資料

- ilearning 作業截圖
- 教授 A-to-Z 影片連結
- media.mp4
- Frank_oral_defense_0725_v7.pptx
- Smart_City_Drone_Law_Enforcement.pdf
- 2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

你要自行補充查找的外部資料

請優先搜尋並引用：

- SUMO 官方文件
- SUMO-RL 官方文件 / 官方 repo
- Gymnasium 官方文件
- Stable-Baselines3 官方文件
- RESCO benchmark
- traffic signal control 相關正式論文
- 必要時可補充 OpenReview、arXiv、官方 GitHub repo

引用品質要求

- 優先用官方與 primary sources
- 不要用低品質部落格當主要依據
- 需要有 citation
- 若是較新的工具或文件，必須查最新資訊
- 不確定的地方要明講，不要硬猜

七、輸出格式要求

1. 請用清楚的 markdown 結構輸出
2. 每個大段落要有標題
3. 提供表格時請真的有比較價值，不要為了表格而表格
4. 有些內容要「可直接拿去交」
5. 有些內容要「可直接複製到 GitHub / PPT / 講稿」
6. 若你能直接產出檔案草稿，也請一併產出
7. 如果內容太長，請你分段清楚輸出，但不要偷省略重要內容

八、最終目標

你這次不是只回答我問題，而是要幫我完成：

- 作業要求解讀
- proposal 主體內容
- related work 方向
- proposed design 規格書
- ppt 章節規劃
- 10 分鐘口說稿
- GitHub 提交內容規劃
- 最終 checklist

請現在開始執行，並先從：

****Step 1：作業需求拆解****

開始。

等你下一回合把這段貼回來後，我就會正式開始執行。

貼上的 Markdown (1).md

檔案

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
 2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 `**"openspec"**`
 3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
- 顯示更多顯示較少

0dbfd7e2-fb91-4798-8da8-e6aa26504b3c.png

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖的時空背景 還有檔案 我現在再補上一次 你可以把它當作溫故知新

然後直接執行以下

用繁體中文給我這個「**15. 給 Antigravity 的 change-01 指令草案**」 我要可以讓 Antigravity 最精準執行任務的「最完美且超高品質的**最佳 Prompt**」

備註：

因為我之前發生過我跟一位 AI Agent 協作

我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec

我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 官方形容的 SDD 開發模式

他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用

這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

你生給我的最佳 Prompt 要包含 OpenSpec 官方的指令 cli api 等等請 Antigravity 實作

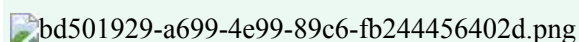
因為是 Antigravity **開工前最佳 Prompt** 要有一槌定音的效果

請用最高規格對待 思考縝密 完備 完美執行生成**最佳 Prompt** 的任務

我要用的是「**官方 OpenSpec 套件**」

顯示更多顯示較少

Phase 2：用 OpenSpec 建立專案真相來源

bd501929-a699-4e99-89c6-fb244456402d.png

<https://github.com/donuop35/ns3-drl-congestion-control/commit/3897594e6e2b7f6f986d84d3ffd5bc537fbd8ac6>

以上 github diff 是 Antigravity 做完「Phase 2：用 OpenSpec 建立專案真相來源 OpenSpec change 01：project-charter」

請幫我驗收是否合格

並且附圖是 Antigravity 請我回答的問題，請幫我一併確認並一一回覆
顯示更多顯示較少

驗收結論：change-01 合格，可批准進入 change-02 的 proposal 階段

我檢查你提供的 commit diff 後，判斷 **OpenSpec change-01：project-charter** 可以驗收通過。它已完成 change-01 應該完成的核心任務：題目凍結、scope / non-goals 凍結、技術方向凍結、baseline / metrics 凍結、MDP 初版定義、risk register，以及 downstream change map。commit 內容也明確寫入「不寫 code、不跑 ns-3、不建 ns3-gym、不啟動 DQN/PPO」等治理規則，符合 change-01 只做 project charter 的邊界。

[change-01: complete project cha...](#)

但我會把這次驗收定義為：**Artifact-level 合格、可進入 change-02 proposal**；CLI 執行證據仍以 **Antigravity 本機回報為準**。

也就是說，GitHub diff 已經顯示它有把官方 OpenSpec 使用證據、`openspec --version = 1.4.1`、`.agent/skills`、`.agent/workflows`、`openspec status --change "project-charter"` 等檢查寫進文件，但我無法從 GitHub diff 直接執行驗證 Antigravity 本機環境。這不是阻塞點，只要你在回覆中要求它保留 command output 或截圖即可。

一、逐項驗收 Antigravity 提出的 6 個確認問題

1. 題目是否正確（中文 / 英文 / GitHub 版本）

結論：正確，通過。

可確認如下：

項目	驗收結果
中文題目	以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
英文題目	Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
GitHub / README 簡版	DRL-Based Congestion Control over a Bottleneck Link

這三個版本都有出現在 change-01 的 project identity 與 charter 文件中，且明確標示不得任意更改。

[change-01: complete project cha...](#)

給 Antigravity 的回覆：

題目正確，中文、英文、GitHub / README 簡版皆接受並正式凍結。後續所有 README、PPT、proposal、OpenSpec changes 都必須沿用此題目，不得自行更名。

2. Scope 是否可接受 (In / Out / Non-Goals)

結論：可接受，通過。

目前 scope 設定是合理的，而且有把本專題最重要的風險全部擋住：

In Scope 包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC baseline
- BBR strongly preferred
- throughput / RTT / loss / utility evaluation

Out of Scope / Strict Non-Goals 也有清楚禁止：

- IPFS implementation
- QUIC implementation
- Linux kernel TCP modification
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol

這符合我們 Phase 0 / Phase 1 定下的「**能做完優先**」策略。commit 中也明確寫入 strict governance rules, 例如不得在 change-01 寫 code、不得啟動 ns-3、不得建立 ns3-gym environment、不得實作 IPFS / QUIC 等。

[change-01: complete project cha...](#)

給 Antigravity 的回覆：

Scope 可接受。In Scope / Out of Scope / Strict Non-Goals 皆通過。請後續 change-02 繼續嚴格遵守：只做 ns-3 baseline benchmark proposal，不得提前進入 ns3-gym、DQN、IPFS、QUIC、multi-agent 或 multi-path。

3. MDP 初版是否接受 (state 4-dim / action Discrete(3) / reward 公式)

結論：接受，但標註為 MVP 初版，可在 change-03 smoke test 後微調。

目前 MDP 初版是：

- **State / Observation** : [throughput_norm, rtt_norm, loss_rate, cwnd_signal_norm]
- **Action** : Discrete(3) = {decrease, keep, increase}
- **Reward** : $r_t = \alpha \cdot \text{throughput}_t - \beta \cdot \text{RTT}_t - \gamma \cdot \text{loss}_t$

這個版本可以接受，原因是它符合 MVP 最小可行原則：state 維度不大、action 是離散動作、reward 不只看 throughput。SB3 的 DQN 文件明確支援 **Discrete** action space，因此用 DQN 作為第一版 MVP 是合理的；PPO 可保留為後續 extension。  [Stable Baselines3 Docs+1](#)

要注意的是：cwnd_signal_norm 是否能直接從 ns3-gym 讀取或控制，還需要到 **change-03-opengym-env** 才能驗證。所以目前應該接受為 **MDP initial abstraction**，不要把它寫死成一定要直接讀寫 cwnd。若後續發現 cwnd 不好操作，可以降階為 sending-rate-like control signal 或 congestion indicator。

給 Antigravity 的回覆：

MDP 初版接受。State 4-dim、Action Discrete(3)、Reward 公式皆可作為 MVP 初版。但 cwnd_signal_norm 目前視為 cwnd-like / congestion-control abstraction，是否能直接讀寫 cwnd 留到 change-03 smoke test 驗證。若不可行，可降階為 sending-rate-like control 或 congestion indicator，不影響 change-02 啟動。

4. Baseline 選擇是否接受 (NewReno + CUBIC 必做，BBR 非 blocking)

結論：接受，通過。

這是正確的 baseline 策略：

- **NewReno**：必做，經典 loss-based TCP baseline。
- **CUBIC**：必做，現代常見 TCP baseline；ns-3 文件也說明 CUBIC 是 Linux、macOS、Windows 常見預設類型之一。  [NSNAM](#)
- **BBR**：strongly preferred，但不 blocking；ns-3 官方文件已有 **TcpBbr**，也說明其基於 bottleneck bandwidth 與 RTT 估計來控制 inflight data，因此適合作為對照組，但若整合成本過高，不應阻塞 MVP。  [NSNAM](#)

ns-3 官方文件也明確展示 NewReno 可透過 **TcpNewReno** 設定，因此 NewReno / CUBIC / BBR 這組 baseline 合理。  [NSNAM+2](#)

給 Antigravity 的回覆：

Baseline 選擇接受。NewReno + CUBIC 為 change-02 必做 baseline；BBR 為 strongly preferred，但非 blocking。若 BBR 在目標 ns-3 版本中整合成本過高，可移至 optional / future work，不得阻塞 change-02 完成。

5. Risk register 是否充分 (R-01 ~ R-12)

結論：充分，通過。

R-01 到 R-12 已涵蓋目前最關鍵風險：

- ns3-gym 安裝 / 版本相容
- ns-3 logging 不完整
- BBR 版本相依
- DQN 不收斂
- reward 學歪
- Antigravity 自行擴題
- 題目被誤解為 IPFS 實作
- DRL 未超越 baseline
- 圖表不足以支撐簡報
- OpenSpec 文件與實作脫節
- 使用假 OpenSpec / 模擬 OpenSpec
- 未使用官方 CLI 卻宣稱完成 OpenSpec

我特別肯定 R-11 / R-12，因為你前面明確遇過「AI Agent 模擬 OpenSpec workflow」的問題。現在 risk register 已經把「fake OpenSpec / simulated OpenSpec workflow」列為最高風險，且要求若發現未使用官方 @fission-ai/openspec 套件就立即停止，這是正確的治理設計。

[change-01: complete project cha...](#)

給 Antigravity 的回覆：

Risk register 充分，R-01 ~ R-12 接受。特別是 R-11 / R-12 對「假 OpenSpec / 未使用官方 CLI」的處理是必要且正確的。後續每個 change 都必須保留 Scope Compliance 與 OpenSpec Verification，不得移除。

6. 是否有任何 Open Questions 需要在進入 change-02 前決策

結論：有 4 個 Open Questions，但都不阻塞 change-02 的 /opsx:propose ns3-baseline-benchmark。

OQ-01：Node.js v20.11.1 < 20.19.0

OpenSpec 官方 README 的 Quick Start 寫明需要 Node.js 20.19.0 或以上，且安裝方式為 `npm install -g @fission-ai/openspec@latest`，再執行 `openspec init`。 

決策：不阻塞 change-02 proposal，但建議儘快升級。

change-02 目前只是啟動 /opsx:propose ns3-baseline-benchmark，若 OpenSpec CLI 目前可用，可以先進入 proposal 階段；但在進入真正 apply / implementation 前，建議升級 Node.js 至官方要求版本，避免後續

OpenSpec workflow 或 agent instructions 出現版本問題。

回覆 Antigravity：

OQ-01 不阻塞 change-02 proposal。可以先啟動 `/opsx:propose ns3-baseline-benchmark`。但請在 change-02 進入 implementation 前，優先升級 Node.js 至 OpenSpec 官方要求的 20.19.0+，或在 README / risk register 保留目前 WARN 與可用狀態紀錄。

OQ-02：ns3-gym 與最新 ns-3 版本的相容性

ns3-gym 官方 README 說明它是整合 OpenAI Gym 與 ns-3，用於 networking RL research；但它也有版本與 framework 相容性注意事項，例如 README 中提到新版 Gym 介面可能需要使用 `ns3env.Ns3Env()`。  [GitHub](#)

決策：不阻塞 change-02，因為 change-02 是 baseline benchmark，不需要 ns3-gym。

ns3-gym 相容性應留到 change-03 解決。change-02 只要專注於 ns-3 baseline：NewReno / CUBIC / BBR、logging、metrics、scenario config。

回覆 Antigravity：

OQ-02 不阻塞 change-02。ns3-gym 相容性留到 change-03-opengym-env 處理。change-02 只需定義與執行 ns-3 baseline benchmark，不得提前串 ns3-gym。

OQ-03：BBR 是否在目標 ns-3 版本支援

ns-3 官方文件已有 `TcpBbr`，並提供設定方式與相關 example，表示 BBR 在目前 ns-3 TCP model 中是有正式文件支撐的 baseline 選項。  [NSNAM](#)

決策：BBR 維持 strongly preferred，但非 blocking。

change-02 必須以 NewReno + CUBIC 完成 baseline 的最低驗收；BBR 若能順利整合就納入，若卡超過原本 risk register 定義的時間上限，移到 optional / future work。

回覆 Antigravity：

OQ-03 不阻塞 change-02。BBR 維持 strongly preferred，但不是 blocking dependency。change-02 必須先確保 NewReno + CUBIC baseline 完成；BBR 若整合成本過高，移至 optional / future work 並記錄原因。

OQ-04：cwnd 是否可在 ns3-gym 中直接讀寫

決策：不阻塞 change-02；留到 change-03 smoke test。

change-02 是 baseline benchmark，不涉及 RL action implementation，也不需要決定 cwnd 讀寫方式。change-03 才需要驗證：

- 是否能直接讀 cwnd
- 是否能直接控制 cwnd
- 或是否改用 sending rate / application-level rate limiter / congestion indicator 作為 action abstraction

回覆 Antigravity：

OQ-04 不阻塞 change-02。cwnd 讀寫能力留到 change-03 smoke test 驗證。現階段 MDP 中的 cwnd_signal_norm 可先視為 cwnd-like congestion-control signal；若直接讀寫 cwnd 不可行，可改用 sending-rate-like control 或 congestion indicator。

二、我給你的正式驗收判定

驗收結果

項目	結果
題目	通過
Scope / Non-Goals	通過
MDP 初版	通過，但 cwnd 細節留到 change-03
Baseline	通過
Risk register	通過
Open Questions	不阻塞 change-02
是否可進入 change-02	可以
建議狀態	Approve with minor notes

小提醒：兩個 minor notes

Minor note 1：OpenSpec path 命名

目前 commit 中使用的是 `openspec/changes/project-charter/`，而不是我們一開始文字上說的 `openspec/changes/01-project-charter/`。如果這是官方 `/opsx:propose` 產生的 change name，且 `openspec status --change "project-charter"` 正常，那可以接受；後續只要在文件中持續稱它為 **change-01：project-charter** 即可。

Minor note 2：Node.js 版本

OpenSpec 官方要求 Node.js 20.19.0+。目前 Antigravity 記錄為 Node.js v20.11.1，雖然功能暫時正常，但建議在進入真正 implementation 前升級，以免後續 `/opsx:apply`、agent workflow 或 update 出現版本不一致問題。  [GitHub](#)

三、你可以直接貼給 Antigravity 的回覆

以下這段可以直接回給 Antigravity：

我已驗收 change-01-project-charter，結論是：通過，可進入 change-02 的 proposal 階段。

逐項確認如下：

1. 題目正確並正式凍結：

- 中文題目：以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
- 英文題目：Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
- GitHub / README 簡版：DRL-Based Congestion Control over a Bottleneck Link

2. Scope 可接受：

In Scope / Out of Scope / Strict Non-Goals 皆通過。後續不得自行擴張到 IPFS、QUIC、kernel-level TCP、real Internet deployment、multi-agent、multi-path、large-scale topology 或 production-level protocol。

3. MDP 初版接受：

state 4-dim、action Discrete(3)、reward 公式皆接受作為 MVP 初版。

但 cwnd_signal_norm 目前視為 cwnd-like / congestion-control abstraction；是否能直接讀寫 cwnd 留到 change-03 smoke test 驗證。若不可行，可改為 sending-rate-like control 或 congestion indicator。

4. Baseline 選擇接受：

NewReno + CUBIC 為 change-02 必做 baseline。

BBR 為 strongly preferred，但非 blocking。若 BBR 整合成本過高，移到 optional / future work，不得阻塞 change-02。

5. Risk register 充分：

R-01 ~ R-12 接受。特別保留 R-11 / R-12：不得使用假 OpenSpec，不得模擬 OpenSpec workflow，不得未使用官方 CLI 卻宣稱完成 OpenSpec change。

6. Open Questions 決策：

- OQ-01 Node.js v20.11.1 < 20.19.0：不阻塞 change-02 proposal，但建議在進入 implementation 前升級到官方要求版本。
- OQ-02 ns3-gym 相容性：不阻塞 change-02，留到 change-03 處理。
- OQ-03 BBR 支援：不阻塞 change-02，BBR strongly preferred but non-blocking。
- OQ-04 cwnd 讀寫能力：不阻塞 change-02，留到 change-03 smoke test 驗證。

批准你啟動：

```
/opsx:propose ns3-baseline-benchmark
```

注意：目前只允許啟動 change-02 的 proposal 流程，不得直接進入 implementation，不得執行 ns-3 benchmark，不得寫 baseline code，除非 change-02 proposal / design / tasks / specs 經我驗收通過。

四、最終建議

你可以批准 Antigravity 進入：

```
/opsx:propose ns3-baseline-benchmark
```

但這只是啟動 **change-02 proposal**，不是讓它開始寫 ns-3 code。

下一次你要驗收的重點應該是：change-02 是否正確產出 **baseline benchmark 的 proposal / design / tasks / specs**，並且是否仍然遵守「baseline 先行、不碰 DRL、不碰 ns3-gym、不碰 DQN」的邊界。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/dea78ef0eb3f19705a18b304b6b3b3b5dc7cebe5>

請看此 github repo

我已經請 Antigravity 再做一次 Change 01 Revisions 的修正

請幫我驗收是否完善

並判斷期末專案是否可推進到 Change 02
顯示更多顯示較少

先註：你先前上傳的部分附件系統顯示已過期；本次我改以你提供的 GitHub commit diff 與官方文件做驗收，不影響這次判斷。

驗收結論：Change 01 revisions 已完善，可推進到 Change 02

我判斷這次 **Change 01 Revisions** 合格，可以正式確認 **change-01 完成**。這個 commit 已針對上次我指出的幾個關鍵點完成修正：Node.js 版本風險被寫入 README 與 design、router-based bottleneck topology 的 wording 修正、**cwnd** fallback rule 補上、README 移除非正式 YouTube embed、成功條件改成分階段未完成項目，而不是誤標全部完成。commit message 也明確標示這次包含「change-01 revisions」與「change-02 ns3-baseline-benchmark proposal complete」。

[change-01: revisions \(Node.js r...](#)

我的判定是：

驗收項目	結論
Change 01 revisions	通過
題目與 README 狀態	通過
Scope / Non-goals 修正	通過
MDP cwnd fallback	通過
Node.js 版本風險註記	通過，但 implementation 前仍建議升級
Change 02 proposal 是否已建立	已建立，可進入 Change 02 驗收 / implementation 前準備
是否可推進到 Change 02	可以
是否可直接進入 Change 02 implementation	可以準備，但建議先要求 Antigravity 回報 change-02 proposal artifacts 供你最後確認

1. Change 01 Revisions 驗收

1.1 README 狀態修正：通過

原本 README 狀態是 Change 01 in progress，且放了一個非正式 YouTube 影片 embed；這次已修正為：

- Change 01：Approved
- Change 02：In Progress
- Demo Video：TODO，正式影片留到 Change 05 補上

這是正確修正。Proposal / demo / final video 不應該在目前階段用非正式影片 placeholder 造成混淆。commit diff 已顯示 README 移除原本 YouTube embed，改成正式 demo video TODO。

[change-01: revisions \(Node.js r...](#)

判定：通過。

1.2 Node.js 版本風險：通過，但 implementation 前仍建議升級

這次已把 Node.js v20.11.1 低於 OpenSpec 官方要求 v20.19.0+ 的問題寫入 README 與 design。OpenSpec 官方 README 明確寫 Quick Start 需要 Node.js 20.19.0 或以上，並使用 `npm install -g @fission-ai/openspec@latest` 與 `openspec init` 初始化；所以這個 warning 必須留下。 

目前可以接受它作為 **non-blocking warning**，因為 Antigravity 已記錄 OpenSpec v1.4.1 可用，且目前只是在 change proposal / spec 階段。

但在 Change 02 真正進入 implementation 前，我建議你二選一：

1. 要求 Antigravity 升級 Node.js 到 20.19.0+。
2. 由你明確給 waiver，允許它在 v20.11.1 有 WARN 的狀態下繼續，但需保留風險紀錄。

判定：通過，但 implementation 前需升級或明確 waiver。

1.3 Topology wording 修正：通過

之前「large topology > 2 nodes」的寫法會誤傷合理的 ns-3 router-based bottleneck topology。這次已改成：

- 允許 single bottleneck path。
- 允許合理的 ns-3 router-based bottleneck topology。
- 禁止 multi-bottleneck。
- 禁止 multi-path。
- 禁止 multiple sender/receiver groups。
- 禁止 large-scale topology。

這個修正很重要，因為 ns-3 做 bottleneck 常見合理形式就是 sender → router → receiver 或 sender → router → router → receiver；只要瓶頸是單一瓶頸鏈路，就不違反 single bottleneck path。commit diff 已補上 router-based topology allowed，並明確禁止 multi-bottleneck / multi-path / multi-sender-receiver。

[change-01: revisions \(Node.js r...](#)

判定：通過。

1.4 MDP cwnd fallback rule：通過

這次已把原本過於硬的 `cwnd_signal_norm` 改成：

`cwnd_or_rate_signal_norm`

並新增 fallback rule：

- 優先使用 `cwnd_signal_norm`
- 若 Change 03 smoke test 發現無法穩定取得或控制 `cwnd`
- 改為 `sending_rate_signal_norm` 或等效 congestion-control abstraction
- 必須在 Change 03 design.md 記錄 Decision Record，並等待 Spec Owner 確認

這是非常正確的設計。因為 `cwnd` 能否直接讀寫屬於 ns3-gym / ns-3 integration 細節，不應該在 Change 01 寫死。現在改成 `cwnd-like` / `rate-like` abstraction，能保留研究方向，又不會讓 implementation 被 `cwnd` API 卡死。commit diff 已補上 `cwnd_or_rate_signal_norm` 與 fallback procedure。

[change-01: revisions \(Node.js r...](#)

判定：通過。

1.5 Success Definition 修正：通過

原本有一個問題：一些其實屬於 Change 02 / 03 / 04 / 05 的成果被標成 [x]，看起來像已完成。這次已修正為：

- Change 01 已完成：研究方向清楚、不偏離凍結題目
- Change 02 目標：Baseline 可重現執行、產出 CSV log
- Change 03 目標：ns3-gym environment 可 reset / step
- Change 04 目標：DQN 訓練腳本與 reward curve
- Change 05 目標：DRL vs baseline 圖表、README 重現、demo script

這個修正讓 project charter 更誠實，也避免 proposal 階段誤宣稱已有實驗成果。commit diff 已將原本全部打勾的成功項目改為分 change 的未完成目標。

[change-01: revisions \(Node.js r...](#)

判定：通過。

2. Change 02 是否可以開始

結論：可以進入 Change 02，但要分清楚兩件事

目前這個 commit 已經不只是修 Change 01，它還新增了 **Change 02：ns3-baseline-benchmark 的 proposal / design / tasks / specs**。從 diff 來看，Change 02 的方向大致正確：它明確定義了只做 ns-3 baseline，不碰 ns3-gym、不建立 RL environment、不訓練 DRL agent，並且把 NewReno / CUBIC 設為必做、BBR 設為 strongly preferred non-blocking。

[change-01: revisions \(Node.js r...](#)

但我要拆成兩個層級判斷：

問題	判斷
是否可從 Change 01 推進到 Change 02 ？	可以
是否可接受 Change 02 proposal 已開始 ？	可以
是否可直接讓 Antigravity 開始 Change 02 implementation ？	建議先做一次 Change 02 proposal 驗收，再放行 <code>/opsx:apply</code>

也就是說，**Change 01 已經沒有阻塞點**；但因為 Antigravity 已經把 Change 02 proposal 也放進同一個 commit，我建議下一步不要直接讓它寫 ns-3 code，而是先請它回報：

change-02 ns3-baseline-benchmark proposal / design / tasks / specs 已完成，請 spec owner 驗收是否可以 `/opsx:apply`。

你再讓我幫你驗收一次 Change 02 artifacts，就可以比較安全地進入 implementation。

3. Change 02 proposal 初步檢查

我先做一個初步判斷：**Change 02 proposal** 方向是對的，但正式 apply 前有 3 個 minor issues 建議先修或確認。

3.1 Change 02 Goals / Non-Goals：通過

Change 02 的 goals 包含：

- 建立 single bottleneck topology
- 執行 NewReno baseline
- 執行 CUBIC baseline
- BBR available 則納入，否則 optional
- 產出 YAML / JSON config
- 解析 ns-3 log 成 CSV
- 產生 throughput / RTT / packet loss 圖
- 更新 README baseline 執行指令

Non-goals 明確排除：

- 不裝 / 不用 ns3-gym
- 不建立 RL environment
- 不訓練 DRL agent
- 不做 DRL vs baseline
- 不做 multi-bottleneck / multi-path / multi-sender topology

這符合 Change 02 「Baseline 先行」的目的。

change-01: revisions (Node.js r...

初步判定：通過。

3.2 Baseline 選擇：通過

Change 02 仍維持：

- NewReno：required
- CUBIC：required
- BBR：strongly preferred / non-blocking

這是正確的。ns-3 官方文件顯示 NewReno 可透過 `TcpNewReno` 設定；CUBIC 是 `TcpCubic`，且官方說明 CUBIC 是 Linux、macOS、Windows 常見預設 TCP congestion control；BBR 在 ns-3 中有 `TcpBbr`，官方文件也提供 BBR 設定方式與 example。📄 [nsnam.org+2](https://www.nsnam.org+2)

初步判定：通過。

3.3 ns3-gym 留到 Change 03：通過

Change 02 明確寫「不安裝或使用 ns3-gym，留到 Change 03」，這是正確的。ns3-gym 官方定位是把 OpenAI Gym 與 ns-3 整合，用於 RL in networking research；因此它應該在 baseline 完成後才進場，不該混進 baseline benchmark。📄 [GitHub](#)

初步判定：通過。

4. Change 02 正式 implementation 前的 3 個小修建議

Minor issue 1：FlowMonitor XML 要求 byte-for-byte identical 太嚴格

Change 02 specs 中有一段要求相同 random seed 兩次執行要產生：

identical output logs (byte-for-byte equivalent FlowMonitor XML)

這個要求太硬。即使 random seed 固定，XML 中也可能因 metadata、排序、時間戳、浮點輸出格式或不同環境造成非 byte-for-byte 等價。建議改成：

The experiment MUST produce metric-equivalent outputs under the same random seed, within a documented tolerance.

也就是要驗證：

- throughput 差距在容許範圍內

- RTT 差距在容許範圍內
- loss rate 差距在容許範圍內

不要要求 XML byte-for-byte identical。

建議：Change 02 apply 前修正。

Minor issue 2：Utility score 權重先不要寫死成最終標準

Change 02 specs 暫定：

```
utility = throughput_norm - 0.1 * rtt_norm - 10.0 * loss_rate
```

這可以當初版，但不宜在 Change 02 就被視為 final utility。因為 normalization 與權重應該到 Change 04 / Change 05 配合 DRL evaluation 再正式定案。

建議改成：

```
utility_score = preliminary composite score for baseline visualization only.  
Weights are provisional and may be revised in Change 04 / Change 05 with Spec Owner approval.
```

建議：Change 02 apply 前補註。

Minor issue 3：Change 02 tasks 已經進入很細的 implementation command，正式 apply 前要確認這就是你要的粒度

Change 02 tasks 目前列了很多 implementation 細節，例如建立 `bottleneck_topology.cc`、`parse_baseline.py`、`plot_baseline.py`、執行 scenario、產出 CSV 與 figures。這些內容符合 Change 02，但它已經不是純 proposal，而是可執行 implementation checklist。

這不是錯，但代表下一步如果你批准 `/opsx:apply`，Antigravity 就會開始真正寫 ns-3 / Python analysis code。你要確認你已準備好讓它進入 Baseline 先行實作。

建議：先讓它修完上面兩個 minor issues，再批准 Change 02 implementation。

5. 最終判斷

是否完善？

Change 01 revisions：完善，通過。

這次修正已經把上次所有關鍵問題補齊：

- Node.js warning 有寫
- topology wording 合理化

- cwnd fallback 有寫
- success definition 分階段
- Open Questions 已標示 non-blocking
- README 移除非正式影片
- Change 01 review items 幾乎全完成，只剩 12.6 等你最後確認

是否可推進到 Change 02 ?

可以。

更精準地說：

我批准 Change 01 完成，也同意專案推進到 Change 02。

但 Change 02 若要進入 implementation，建議先修正或確認：

1. 不要求 FlowMonitor XML byte-for-byte identical。
2. Utility score 權重是 provisional，不是 final。
3. Node.js 版本升級或由你明確 waiver。

6. 你可以直接貼給 Antigravity 的回覆

我已驗收 commit dea78ef0eb3f19705a18b304b6b3b3b5dc7cebe5。

結論：

Change 01 revisions 通過，project-charter 可視為正式完成。

期末專案可推進到 Change 02：ns3-baseline-benchmark。

已確認通過項目：

1. README 狀態已修正：Change 01 Approved、Change 02 In Progress、Demo Video 改為 TODO。
2. Node.js v20.11.1 < 20.19.0 的 OpenSpec 版本風險已補入 README / design。
3. Topology wording 已修正：允許合理的 ns-3 router-based single bottleneck topology；禁止 multi-bottleneck、multi-path、multi-sender/receiver groups、large-scale topology。
4. MDP 第 4 維已修正為 cwnd_or_rate_signal_norm，並補上 cwnd fallback rule。
5. Success Definition 已改為分 Change 階段，不再誤標尚未完成的 baseline / ns3-gym / DQN / demo 成果。
6. OQ-01 ~ OQ-04 已標示為不阻塞 Change 02。

批准：

- Change 01 正式完成。
- 可以推進到 Change 02。

但在 Change 02 進入 implementation 前，請先完成以下小修：

A. Reproducibility requirement 不要要求 FlowMonitor XML byte-for-byte identical。
請改成 metric-equivalent outputs under the same random seed, within documented tolerance。

B. Utility score 權重目前只能視為 preliminary / provisional。
請補註：Change 02 的 utility score 只用於 baseline visualization；最終 reward / utility 權重可在 Change 04 / 05 經 spec owner approval 調整。

C. Node.js 版本：

目前 v20.11.1 低於 OpenSpec 官方要求 v20.19.0+。

我允許 Change 02 proposal 繼續，但在 Change 02 implementation 前，請升級 Node.js 到 20.19.0+，或再次向我請求 waiver。

完成上述修正後，再回報：

「Change 02 proposal revisions complete, ready for spec owner review before /opsx:apply。」

注意：

目前仍不得直接開始 ns-3 coding / benchmark execution，除非 Change 02 proposal / design / tasks / specs 經 spec owner 最終驗收通過。

7. 下一步建議

下一步不是直接進入 ns-3 實作，而是請 Antigravity 先針對 Change 02 做這三個小修。修完後，你再把 commit 丟給我，我會幫你做 **Change 02 proposal 驗收**。如果 Change 02 proposal 通過，才批准它執行：

/opsx:apply

這樣節奏是最穩的：**Change 01 已完成** → **Change 02 proposal 小修** → **Change 02 proposal 驗收** → **Change 02 implementation**。

6. 你可以直接貼給 Antigravity 的回覆

請您確認 Change 02 的以下事項：

1. proposal.md：Why / Capabilities / Impact / Acceptance Criteria 是否可接受
2. design.md：D-04 BBR decision gate / D-01 router-based topology 是否可接受
3. spec.md：specs 中的 SHALL/MUST 要求是否正確反映需求
4. OQ-02.01：ns-3 目標版本是否有偏好（3.35? 3.36? 最新穩定版？）
5. Node.js 升級：是否要在 Change 02 implementation 前升級至 v20.19.0+，或授予 waiver？

以上這是 Antigravity 做完 Change 01 要請我確認的事項

請幫我回答

並收攏進原本要貼給 Antigravity 的回覆裡

我要一併回饋給 Antigravity

顯示更多顯示較少

先註：你先前部分上傳附件已顯示過期；這次我以目前 GitHub diff、官方 OpenSpec / ns-3 / ns3-gym 文件來判斷，不影響這次 Change 02 問題回覆。

結論

你可以回覆 Antigravity：Change 01 revisions 已通過，Change 02 proposal 方向可接受，但在進入 Change 02 implementation 前，請先依照以下決策做小修。

最重要的決策有兩個：

1. **ns-3 目標版本指定為 ns-3.40**，不要先用最新穩定版。理由是 ns3-gym 官方 README 的安裝流程直接使用 `ns-allinone-3.40`，並切到 `app-ns-3.36+` branch；雖然 ns-3 官方 releases 頁面目前列到 ns-3.47，但本專案後面 Change 03 要接 ns3-gym，因此以 ns-3.40 當 baseline 與後續銜接版本，風險最低。 [GitHub+1](#)
2. **Node.js 請在 Change 02 implementation 前升級到 v20.19.0+**，不授予 **waiver**。OpenSpec 官方明確要求 Node.js 20.19.0 or higher，且官方流程是安裝 `@fission-ai/openspec` 後執行 `openspec init /opsx:propose`；既然你已經明確要求「使用官方 OpenSpec 套件」，這裡不要留下版本風險。 [GitHub](#)

逐項回覆 Antigravity 的 5 個確認事項

1. proposal.md：Why / Capabilities / Impact / Acceptance Criteria 是否可接受

結論：可接受，但需補兩個小修後再進入 implementation。

proposal.md 的方向是對的：Change 02 的核心是建立 ns-3 baseline benchmark，不碰 RL、不碰 ns3-gym、不碰 DQN；這符合前面定義的「Baseline 先行」。目前 proposal 已寫明 Change 02 會建立 single bottleneck topology、執行 NewReno / CUBIC，BBR 若支援則納入，並產出 CSV log、baseline figures、summary table，這些都符合需求。diff 中也清楚列出 Change 02 的 goals / non-goals。

[change-01: revisions \(Node.js r...](#)

但 Acceptance Criteria 裡有兩點要修：

- **不要要求 FlowMonitor XML byte-for-byte identical**。ns-3 FlowMonitor 官方文件說明它能輸出 XML，也說明 FlowMonitor 統計會受 packet tracking、simulation stop timing、in-flight packets 等因素影響；因此驗收應該看 metrics 是否在 tolerance 內一致，不要要求 XML byte-for-byte 完全相同。
[nsnam.org](#)
- **Utility score 權重只能是 provisional**。Change 02 是 baseline benchmark，不是最終 DRL reward tuning；`throughput_norm - 0.1 * rtt_norm - 10.0 * loss_rate` 可作為 baseline visualization 的 provisional utility，但不能在這階段凍結成最終 reward / utility。

回覆結論：proposal.md 可接受，但需修正 reproducibility 與 utility wording。

2. design.md：D-04 BBR decision gate / D-01 router-based topology 是否可接受

結論：兩者都可接受。

D-01 router-based topology

接受。

router-based topology 是合理的 ns-3 single bottleneck 實作方式，只要專題範圍仍然是 **single bottleneck path**，中間有 router 並不違反 scope。這次修正已把原本容易誤傷的「超過 2 nodes 不行」改成允許合理 router-based bottleneck，並禁止 multi-bottleneck、multi-path、multiple sender/receiver groups，這是正確修正。

[change-01: revisions \(Node.js r...](#)

D-04 BBR decision gate

接受。

BBR 維持 **strongly preferred but non-blocking** 是正確策略。ns-3 TCP models 文件列出 BBR、CUBIC、NewReno 等 TCP congestion control variants，適合作為 baseline 候選；但本專題 MVP 不應讓 BBR 阻塞 NewReno / CUBIC 的基礎 benchmark。

回覆結論：D-01 / D-04 都接受。

3. spec.md：specs 中 SHALL / MUST 要求是否正確反映需求

結論：大致正確，但有兩條 MUST 要調整。

目前 spec.md 大多數 SHALL / MUST 是合理的，包含：

- 必須建立 configurable single bottleneck topology
- 必須執行 NewReno / CUBIC
- BBR 可用則執行，不可用則 non-blocking optional
- 必須輸出 throughput / RTT / loss CSV
- 必須產生 baseline figures
- 必須禁止 RL / ns3-gym / DQN code 進入 Change 02

這些都符合 Change 02 需求。

[change-01: revisions \(Node.js r...](#)

但請改兩點：

1. Reproducibility 不要寫 byte-for-byte equivalent FlowMonitor XML

改成：

The same scenario with the same random seed SHOULD produce metric-equivalent outputs within documented tolerance.

2. Utility score 權重不要寫成 final MUST

改成：

utility_score SHALL be included as a preliminary baseline visualization metric. The formula and weights are provisional and may be revised in Change 04 / Change 05 with Spec Owner approval.

回覆結論：spec.md 可接受，但以上兩條需修正後才適合 apply。

4. OQ-02.01：ns-3 目標版本是否有偏好

決策：指定 ns-3.40。

理由：

- ns-3 官方 releases 頁面目前列到 ns-3.47，代表「最新穩定版」確實存在；但本專題不是只做 baseline，後續還要接 ns3-gym。 nsnam.org
- ns3-gym 官方 README 的安裝流程直接下載 **ns-allinone-3.40**，並在 ns-3.40 的 **contrib** 下 clone ns3-gym、checkout **app-ns-3.36+** branch。這表示 ns-3.40 是官方 README 明確示範且對本專案 Change 03 最友善的版本。 [GitHub](#)
- ns3-gym App Store 也標示 release 1.0.2 works with ns-3.36，所以 ns-3.40 位於合理相容範圍內。 apps.nsnam.org

所以不要選 3.35 / 3.36 / 最新 3.47。

我的正式決策是：

Change 02 / 03 統一以 **ns-3.40** 作為目標版本。若 Antigravity 發現 ns-3.40 在當前環境無法穩定執行，需回報 Spec Owner，不得自行升到最新版或降到舊版。

5. Node.js 升級：升級或 waiver？

決策：升級，不授予 waiver。

請在 **Change 02 implementation 前** 升級到 Node.js v20.19.0+。

理由是 OpenSpec 官方 Quick Start 明確要求 Node.js 20.19.0 or higher。既然這個專案特別要求使用官方 OpenSpec，而不是模擬 OpenSpec workflow，就不應該在 implementation 階段繼續用低於官方要求的 Node.js 版本。 [GitHub](#)

正式決策：

Change 02 proposal 可以先完成；但在 **/opsx:apply** 或任何 implementation 前，必須升級 Node.js 至 v20.19.0+，並回報 **node -v** 與 **openspec --version**。本次不授予 waiver。

可直接貼給 Antigravity 的完整回覆

我已確認 Change 02 的問題與 Change 01 revisions 狀態。

結論：

Change 01 revisions 通過，project-charter 可視為正式完成。

專案可以推進到 Change 02：ns3-baseline-benchmark。

但 Change 02 在進入 implementation / **/opsx:apply** 前，請先依照以下決策修正 proposal / design / specs。

1. proposal.md：Why / Capabilities / Impact / Acceptance Criteria

可接受。

proposal.md 的方向正確：

- Change 02 聚焦 ns-3 baseline benchmark
- 不碰 RL

- 不碰 ns3-gym
- 不碰 DQN / PP0
- 不做 DRL vs baseline
- NewReno + CUBIC 為 required baseline
- BBR 為 strongly preferred but non-blocking
- 需要輸出 CSV log、baseline figures、summary table

但 Acceptance Criteria 請修正兩點：

A. Reproducibility 不要要求 FlowMonitor XML byte-for-byte identical.

請改成：

The same scenario with the same random seed SHOULD produce metric-equivalent outputs within documented tolerance.

B. Utility score 權重只能是 preliminary / provisional.

請補註：

Change 02 的 utility_score 只作為 baseline visualization metric。公式與權重是 provisional，最終 reward / utility 權重可在 Change 04 / Change 05 經 Spec Owner approval 後調整。

2. design.md:D-04 BBR decision gate / D-01 router-based topology

可接受。

D-01 router-based topology 接受：

- 允許合理的 ns-3 router-based single bottleneck topology
- router-based topology 不違反 single bottleneck path
- 禁止的是 multi-bottleneck、multi-path、multiple sender/receiver groups、large-scale topology

D-04 BBR decision gate 接受：

- NewReno + CUBIC 是 required
- BBR 是 strongly preferred but non-blocking
- 若 BBR 整合成本過高或版本不可用，不得阻塞 NewReno / CUBIC baseline
- BBR 可移至 optional / future work，但必須文件記錄原因

3. spec.md:SHALL / MUST 要求

大致可接受，但請修正兩條：

A. 不要寫：

byte-for-byte equivalent FlowMonitor XML

請改成：

metric-equivalent outputs under the same random seed, within documented tolerance

B. Utility score 不要寫成 final requirement.

請改成：

utility_score SHALL be included as a preliminary baseline visualization metric. The formula and weights are provisional and may be revised in Change 04 / Change 05 with Spec Owner approval.

其餘 SHALL / MUST 方向可接受：

- MUST create configurable single bottleneck topology
- MUST run NewReno and CUBIC
- BBR SHALL run if available, otherwise documented as non-blocking optional
- MUST output throughput / RTT / loss CSV
- MUST generate baseline figures
- MUST NOT include RL / ns3-gym / DQN code in Change 02

4. 0Q-02.01:ns-3 目標版本

正式決策：

使用 ns-3.40 作為 Change 02 / Change 03 的目標版本。

理由：

- 後續 Change 03 需要銜接 ns3-gym

- ns3-gym 官方 README 直接示範使用 ns-allinone-3.40，並搭配 app-ns-3.36+ branch
- 因此 ns-3.40 比最新穩定版更適合本專案的低風險路徑

請不要自行改用 ns-3.35、3.36 或最新穩定版。

若 ns-3.40 在實際環境無法穩定執行，請停止並回報 Spec Owner，不得自行升降版本。

5. Node.js 升級

正式決策：

Change 02 proposal 可以繼續；但在 Change 02 implementation / /opsx:apply 前，必須升級 Node.js 至 v20.19.0+。

本次不授予 waiver。

請在 implementation 前回報：

- node -v
- openspec --version
- 是否仍有 EBADENGINE warning

6. 是否可進入 Change 02

可以推進到 Change 02，但目前只批准 Change 02 proposal revisions。

請先完成以上修正，再回報：

“Change 02 proposal revisions complete, ready for Spec Owner review before /opsx:apply.”

注意：

在我驗收 Change 02 proposal revisions 前，不得開始 ns-3 coding，不得執行 benchmark，不得建立 experiment results，不得寫 analysis script，不得進入 ns3-gym / DQN。

我的建議

你可以把上面整段直接貼給 Antigravity。這樣回覆會同時解決它問你的 5 個確認項，也會維持我們原本治理節奏：

Change 01 完成 → Change 02 proposal 小修 → Change 02 proposal 驗收 → 再批准 /opsx:apply。